

Figure 1: Enter Caption

1 problem

1.specification: input: an array of intervals[start, end] output:non overlapping intervals edge cases 1.empty 2.one interval only 3.no overlapping intervals

2.structure: first we sort all the intervals start from input, then check out if the interval's "end" smaller than next interval's start. if so, we merge two intervals into one, and put this new interval into the output intervals. else, we just put first interval into output intervals. let the second interval to compare the next one.

```

\paragraph3.pseudocode:
input:[start, end].....
output:merged[start,end].....
if interval empty
    return empty
else:
    sort intervals "start"
    for i in n-1
        if current[i].end<=next[i+1].start
            return current[start,end]
        else:
            return new[current.start, next.end]
        end if
    end for
end

```

4.correctness: The intervals in the new arrays will not be overlapped. because the first step, sorting, makes sure that the next interval's start can bigger than the current's start. we will return the empty in the start of programming. if only one interval output, we will directly return it.

5.analysis complexity time: sorting($n \log n$) space: $O(n)$

2 problem

1.specification: input: an mixed array of 0 1 2 output: orderly clear 0 1 2
not have edge cases

2.structure: we can use two method to complete this task. The first method is count the number of 0 1 2, and use a pointer to write them orderly. Secondly, we can use Dutch flag method, which is set three pointers and only search for one time.

```
\paragraph{3.persudocode:}
count:
array[.....]
set three boxes: a,b,c
for i in array
    if i =0, a++
    else if i=1, b++
        else i=2, c++
        x=0
        for i in i<=a,b,c
            array[x]=0,1,2
            x++
```

```
Dutch flag:
set three pointers: left current and right
left=0
current=0
right=array.size
if array[current]==0
    swap(array[left],array[current])
    left++
    current++
else if array[current]==1
    current++
else swap(array[right],array[current])
```

4.correctness: count: we have already set three boxes for 0 1 2, so if we fill it directly based on their number, it will be correct. Dutch flag: the number between current and left will always be 1, the number on the left's left will be 0, we just need to judge the number between current and right. so if current over right, the programming will be ceased.

5.analysis complexity: time: count: $O(n)$ Dutch flag: $O(n)$ space: count: $O(1)$ Dutch flag: $O(n)$

```

Problem2
input:[2, 0, 2, 1, 1, 0]
count sort output:[0, 2, 1, 1, 1, 0]
dutch flag output:[0, 0, 1, 1, 2, 2]
expect (sorted 0/1/2):[0, 0, 1, 1, 2, 2]

```

Figure 2: Enter Caption

```

Problem3
input:[3, 30, 34, 5, 0]
output:9
expect: 9534330

```

Figure 3: Enter Caption

3 problem

1.specification: input: non negative vector output: a string with biggest case edge cases: 1.empty 2.only one number 3.all the zero

2.structure: we can translate all the number in the vector into strings, and compare the sum with each other after double numbers added. So, in this way, we can get a sequence of two numbers. by the way, we also need to use the sort function to help to compare numerous numbers in the group. finally, we need box called result to get these strings together.

```

\paragraph{3.pseudocode:}
if vector empty
    return""
translate numbers into strs
new numbers= many strs
sort( str.begin(),str.end(),) if a+b>b+a,

string result=0
for i in strs
    result=result+str
return result

```

4.correctness: we use the sort function, so it means we will compare these numbers many times to form the final sequence. And judge $a+b > b+a$ make sure that if two numbers with different length, we also can get the biggest result

5.analysis complexity: time: $O(n \log n)$ space: $O(n*d)$ (d means the length of final number)

4 problem

1.specification: input: a vector with many words output: two dimensional vector with group anagram edge cases: 1.empty 2.repeated words

```

Problem4
input:[eat, tea, tan, ate, nat, bat]
output:[[bat], [tan, nat], [eat, tea, ate]]
expect groups:[eat,tea,ate], [tan,nat], [bat]

```

Figure 4: Enter Caption

2.structure: we can translate all the words in the vector into sequence letters, and use the unordered map to gather them with different group. And then we pick up these groups and put them into a new two dimensional group.

```

\paragraph{3.pseudocode:}
set a empty map
for all words in the vector
    translate words into strings
    put string into key
    map<<key
end

set a empty 2 dimensional vector "result"
for all the keys in map
    result<<key
end

```

4.correctness we will translate the different words into letters with definite sequence. so in this way, we can pick up the words with same letters, and put them into new group. it make sure that our group must have the words with same letters.

5.analysis complexity: time: $O(n*L*\log L)$ (L means the length of the longest word) space: $O(n*L)$

5 problem

1.specification: input: two dimensional array output: boolean value edge cases: 1.empty 2.only a long meeting 3.only one meeting input

2.structure: first, we need to get a sequence of all the one dimensional array. so in this way, we compare the last value of ahead array with the first value of the next array. if there is no interval or gaps, even overlapping, this person can not attend all the meetings. the method is same with the problem 1.

```

\paragraph{3.pseudocode:}
if input of meetings empty
    return true
end

```



Figure 5: Enter Caption

```

sort meetings based on 'start'
for i in meetings
    if first meeting.end>next meeting. start
        return false
    end
end for
return true

```

4.correctness: if no conflicts between two edges, we can recognize that person can attend two meetings. we consider the input empty and the case of only one meeting. just like the problem 1 method, we sort the meeting based on first value and compare the edges.

5.analysis complexity: time: $O(n \log n)$ space: $O(1)$

6 problem

1.specification: input:several points(a,b), k number output: k points(a,b)
edge cases: 1.empty 2.the points with same distance

2.structure: 1.sort: just calculate the distance of each points, then pick up the points with shorter distance. output k number points. heap: using the max heap, if the number of these points less than k, just add them in the output. But if the points number more than k, we should throw out the top point of max heap, and add the new one, which is less than top point.

```

\paragraph{3.persudocode:}
sort:
for i in points
    distance=x*x+y*y
end for
sort points based on distance
return k points

heap:
for i in points
    distance=x*x+y*y

```

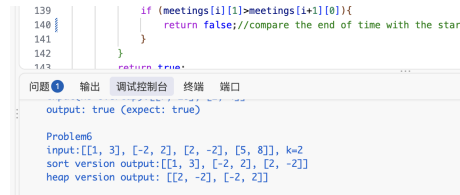


Figure 6: Enter Caption

```

if max heap.size()<k
add
else if distance <maxheap.top()
    maxheap.pop()

pick up all elements in maxheap
return result

```

4.correctness: sort: we use the distance as a regulation to pick up the k points, even the input is empty, wen just output empty. heap: max heap helps me to throw out the biggest point and add the smaller one, so the output is the number in the heap. and just return it, we can get the output vector.

5.analysis complexity: time: sort: $O(n \log n)$ heap: $O(n \log k)$ space: sort: $O(n)$ heap: $O(k)$

7 problem

1.specification: input: a array with numerous numbers output: return the frequently k numbers, package in a new array edge cases: 1. empty 2.the numbers in input less than k

2.structure: we can store the data by using(number, frequency).so we use the sort function to compare the different numbers frequency in the array, and regulate them from high frequency to low frequency. Then we compare the same frequency numbers with numerical value from low to high. All the calculation can be done in sort function. finally, we return the new array with k number.

```

\paragraph{3.persudocode:}
input: a array of numbers, a one dimensional vector
unordered map
for numbers in array
    get the frequency of all numbers

put (number, frequency) into a two dimensional vector

```

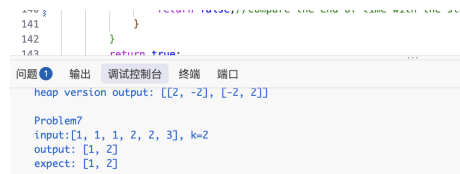


Figure 7: Enter Caption

sort vector based on frequency, numerical values

```

for i in vector
    pick up numerical value part
    put it in the result
end

```

4.correctness by using unordered map, we can easily get the frequency of each number. Then we sort them directly in the new two dimensional vector. finally get the numerical value part, and put them into a new one dimensional vector, that is result.

5.analysis complexity: $O(m \log m)$

8 problem

1.specification: input: several ordered arrays output: on ordered arrays

2.structure: firstly, we can put all arrays' first element with its position information in the min heap, than compare them, pick up the smallest number, throw it into new array. when we pick up the top element of min heap, we will add the next element of it. so we just repeat it, until the min heap is empty.

\paragraph{3.persudocode:}

Min heap <value,array number, number>

```

for i in arrat[i]
    if Arrays[i] empty
        return " "
    else
        Min Heap.push((Arrays[i][0], i, 0))
    end
while Min Heap not empty
    (value,array number, number) ← Min Heap.pop Min()
    Result.append( )
    if this array still has next number, add it

```

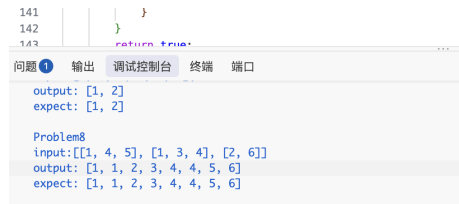


Figure 8: Enter Caption

```

        until min heap empty
    end
end

```

4.correctness: we can use the min heap to get the number in different arrays, from smallest to highest numbers. so in this way, we just need to pick up min number from min heap, to form a new array orderly. even existing the special array with one number or empty, we can use if condition to deal with it.

5.analysis complexity: time: $O(N\log n)$ space: $O(k)$

9 problem

1.specification: input:a unordered array output: kth biggest number in array
no edge case

2. structure: we still can use the min heap to deal with it. if min heap numbers less than k, we add new number from unordered array directly. but, if more than k numbers, we start throw out the top number until only k numbers in min heap.

```

\paragraph{3.persudocode:}
set up a min heap
if min heap.size<k
    add new one
else
    if new number > top number
        min heap.pop
        add new on come in
return min heap.top

```

4.correctness: we keep k number in the min heap, so the top one is kth biggest number. I think it easy to understand. In the min heap, the top one is smallest, so we just need to compare the new one with top one, and kick off the smaller one. when array is empty, the top one in min heap is what I want.

```

expect: [1, 1, 2, 3, 4, 4, 5, 6]

Problem9
input:[3, 2, 1, 5, 6, 4], k=2
output: 5
expect: 5

0.0001 sec 10

```

Figure 9: Enter Caption

5.analysis complexity: time: $O(n \log k)$ space: $O(k)$

10 problem

1.specification: input: a unordered array of numbers output: $i > j$, $\text{value}[i] > \text{value}[j]$

2.structure: first, we can divide this array into two parts, then merge two sorted arrays, if we take a number from right half before left half finished, it means all the numbers which still leave in left half are bigger than it, so they actually form an inversion

3.pseudocode: 1.specification: input: a unordered array of numbers output: number of inversion pairs (i, j) where $i < j$ and $\text{values}[i] > \text{values}[j]$, use long long to hold the count no edge case, but empty/one element array just return 0

structure: this is related to merge sort. when we merge two sorted halves, if we take a number from right half before left half is done, it means all the numbers left in left half are bigger than this one, so they all form an inversion with it. we can count them all at once instead of one by one.

```

\paragraph{3.pseudocode:}
merge sort the array,
compare left and right one by one
if left number <= right number
    put left number in
else
    put right number in
    count += all numbers left in left side
while
answer = left half count + right half count + count from merge step

```

4.correctness: left half and right half are already sorted before we merge, so when we pick up the right number, all the numbers still leave in left side must be bigger than it, because left side is in order. so in this way we can add them all at once. total inversion is just left part add right part, we just need to count the cross part, when they merging.

5. analysis complexity: time: $O(n \log n)$ space: $O(n)$

```
expect: 3  
  
Problem0  
input: [2, 4, 1, 3, 5]  
output: 3  
expect: 3  
  
The program "/Users/gachen/Desktop/美国东北大学ECE/计算机工程基础/EECE7205/Assignments01_jiachen"  
0).
```

Figure 10: Enter Caption